

Assignment 4 - Inverse problems

Rasmus Madsen, Asger Andersen, Benjamin Siddique

January 2026

1 Making the code

1.1 Profiler

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
57.1	3252040366	41	79318057.7	60101863.0	2174	794806860	136637489.9	poll
30.4	1733170480	89	19473825.6	7504.0	1032	830510651	118890130.3	read
5.1	290704718	34	8550138.8	7043.0	1893	257250394	44294321.4	recvmsg
4.6	261157683	2556	102174.4	18098.5	1382	13160040	388098.9	ioctl
2.0	115331843	13	8871680.2	9211440.0	7157297	10764058	1028708.8	writew
0.4	19968318	505	39541.2	4789.0	3336	8884049	472361.2	fopen
0.2	11547874	491	23519.1	1863.0	1272	10572491	477041.9	fclose
0.1	6429757	100	64297.6	64095.0	1202	116176	10925.9	fgets
0.0	2812911	115	24460.1	9337.0	4499	543815	61054.5	mmap64
0.0	1490645	221	6745.0	6963.0	3487	16180	2045.8	open64
0.0	617830	15	41188.7	33122.0	17072	88415	22397.5	sem_timedwait
0.0	495485	7	70783.6	61225.0	46416	98854	20027.8	pthread_create
0.0	405860	26	15610.0	6642.5	2835	78056	22675.0	mmap
0.0	310784	103	3017.3	2685.0	1102	11641	1598.8	write
0.0	299557	238	1258.6	1122.0	1002	24075	1491.7	fcntl
0.0	284371	1	284371.0	284371.0	284371	284371	0.0	fork
0.0	256208	2	128104.0	128104.0	113832	142376	20183.7	pthread_cond_wait
0.0	252521	6	42086.8	7288.5	6191	214681	84564.6	munmap
0.0	232465	2	116232.5	116232.5	79579	152886	51835.9	pthread_join
0.0	157698	33	4778.7	3517.0	1974	12814	2863.9	sendmsg
0.0	145109	14	10364.9	9988.5	4749	19667	4346.2	open
0.0	130734	5	26146.8	17633.0	7905	61514	20962.6	connect
0.0	111775	16	6985.9	5160.0	4087	14487	3456.7	pipe2
0.0	99011	23	4304.8	4608.0	1322	11521	2655.0	putc
0.0	77766	2	38883.0	38883.0	7384	70382	44546.3	pthread_mutex_lock
0.0	75011	7	10715.9	7284.0	4829	25026	7158.9	socket
0.0	72566	9	8062.9	2514.0	1172	43341	13724.4	fwrite
0.0	34115	7	4873.6	4218.0	3427	7434	1542.5	pthread_cond_broadcast
0.0	9196	8	1149.5	1132.0	1032	1352	106.7	dup
0.0	7273	2	3636.5	3636.5	3025	4248	864.8	sigaction
0.0	6893	1	6893.0	6893.0	6893	6893	0.0	memfd_create
0.0	6321	2	3160.5	3160.5	2554	3767	857.7	bind
0.0	5520	1	5520.0	5520.0	5520	5520	0.0	stat
0.0	4578	1	4578.0	4578.0	4578	4578	0.0	pthread_mutex_trylock
0.0	2965	1	2965.0	2965.0	2965	2965	0.0	ftruncate
0.0	2736	2	1368.0	1368.0	1233	1503	190.9	listen

Processing [report3.sqlite] with [/opt/nvidia/hpc_sdk/Linux_x86_64/25.1/profilers/Nsight_Systems/host-linux-x64/reports/cuda_api_sum.py]...

**** CUDA API Summary (cuda_api_sum):**

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
74.8	459670545	20	22983527.3	23856191.0	52217	45583329	23042117.1	cudaMemcpy
13.7	83903247	12174	6892.0	2575.0	2385	1392309	21671.3	cudaLaunchKernel
11.3	69209077	10	6920907.7	103403.0	2574	68462426	21623527.2	cudaMalloc
0.2	1482057	10	148205.7	171825.5	4418	323965	92224.4	cudaFree
0.0	1352	1	1352.0	1352.0	1352	1352	0.0	cuModuleGetLoadingMode

Figure 1: Profiler using erda showing time intensive task like poll and read for the CPU. For the GPU the most time intensive task are cudaMemcpy.

1.2 Code strategy

Our overall strategy for parallelization revolved around decomposing the central step-function into smaller kernels to avoid race conditions (i.e. the kernel_update_eta, kernel_update_u etc). In doing so, we had to reformat multiple nested for-loops and instead index over lists in a row-major fashion. This represented a major challenge in parallelization, as we were dealing with grids of different sizes ($\eta(j, i)$ with dimensions

$[n_y, n_x]$, $u(j, i)$ with $[n_y, n_x + 1]$ and $v(j, i)$ with $[n_y + 1, n_x]$, but using the consistent rule of translating coordinates as $(y, x) \rightarrow (y \cdot N_{rows} + x)$ it was overcome. Further improvements could have been shared memory, drastically reducing the time used on copying memory, see profiler. Read from global memory is significantly slower and as we read the same values from global memory several times, we could for example load the `H_at_u` array into local memory of the block. This is efficient because this is an array we are loading multiple times and not just once. This would be the next step to speedup our code, as well as minimize kernel calls. But using shared memory would probably give the biggest speedup. This is because a global memory fetch is 500 cycles while shared memory is 5 cycles.

2 Weak scaling

To measure weak scaling, where the workload increases along with the available resources, we make use of the `run_tsunami.sh` script, which allows us to use a fraction of the GPU in units of 2 SMPS (With full GPU capacity at 14 SMPS). A scale parameter is passed to the script, which determines the workload of the given problem. Since a doubling of the scale parameter results in a quadrupling of the grid squares and a doubling of the simulation steps, meaning that to increase the workload by a factor N we must choose *scale* to be:

$$\text{scale} = N^{\frac{1}{3}} = \left(\frac{\text{number of smp's}}{2} \right)^{\frac{1}{3}} \quad (1)$$

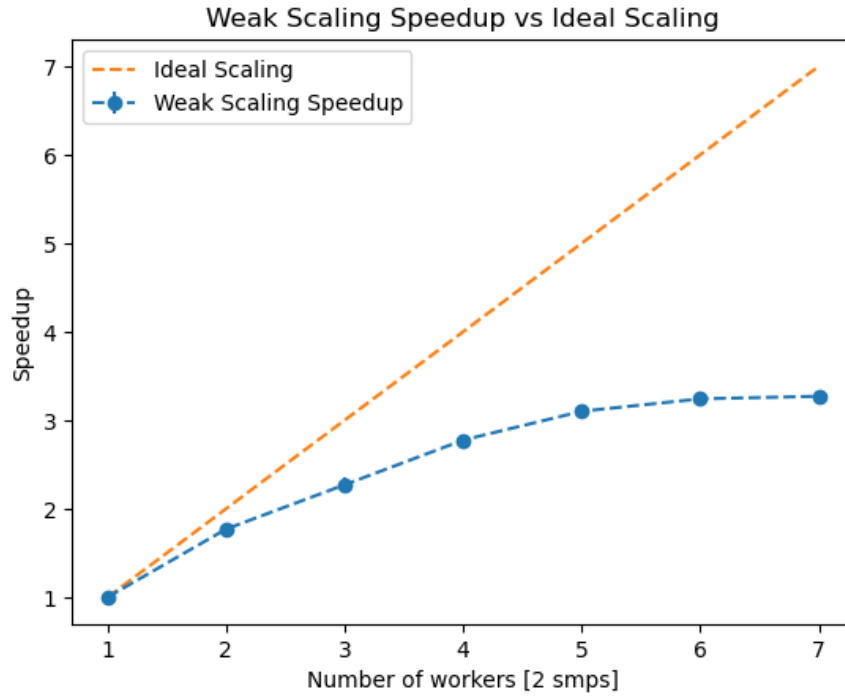


Figure 2

In our code we used blocks of size 16×16 , allowing for 256 threads to run on each block. This is well below the 1024 thread count cap, due to the fact that our SMP's can only handle 2048 threads simultaneously. If we were to create 2 giant blocks from this, it is very likely that both of the blocks would, at some point, be stalled waiting for memory at the exact same time. This is terrible for efficiency, as it would leave the warp scheduler and GPU idle. Instead our choice of $\frac{2048}{256} = 8$ independent blocks for each SMP, almost surely guarantees a consistent flow of warps, avoiding memory latency that would otherwise slow down our simulations. With 8 blocks then, even when two or more blocks are stalled waiting for memory, we may

simply fetch warps from the next block and so on. We could choose an even smaller block size and create more blocks if we wanted to, but it would also create larger overhead and scheduling costs.

To make the workload on the SM's the same we scaled the scale of the simulation appropriately, scaling it with the cubic root since the workload increases by the cube when increasing the scale. We thus ran for the following scales $\{1, 1.26, 1.442, 1.587, 1.71, 1.817, 1.913\}$. Figure 2 clearly shows that the computational speed weak scales. This is evident (look figure 1) due to the fact that most of the time is spend copying memory instead of running the kernels which is why we only see a speed up of 3 instead of 7 for the ideal scaling. Thus as stated earlier optimizing how memory is loaded would certainly give an increase in performance. From the flattening of the graph we can also assume that even if we could access more sm's there would be no additional speedup as we already have saturated the speedup, because of the speed of the code being bounded by memory loading.

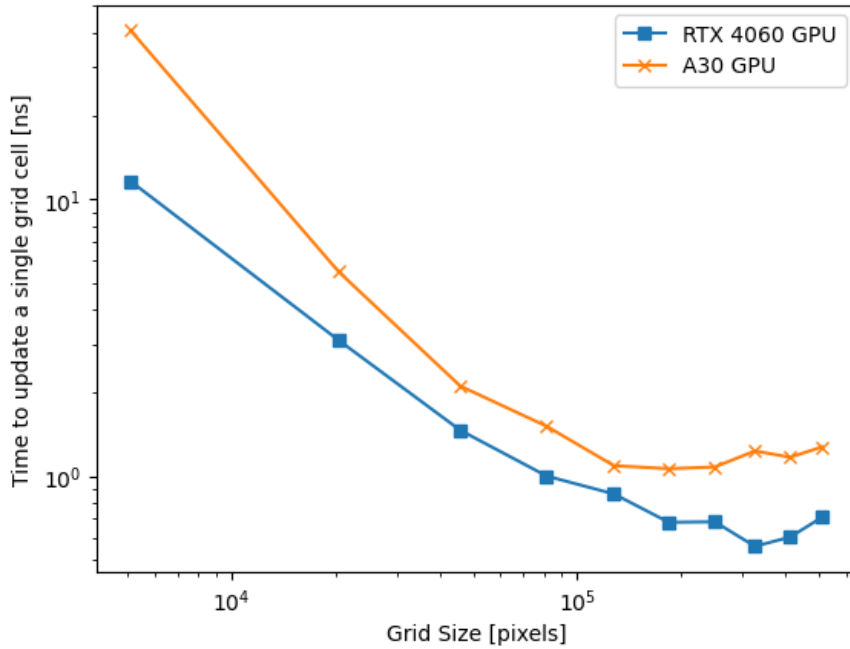


Figure 3: Here we see the average time to update a single grid cell as a function of gridsizes. Two datasets are plotted

In figure 3 we see the average time to update a single grid cell as a function of grid size. Here we plot the A30 GPU performance vs RTX 4060 GPU. We ran for various different grid sizes corresponding to scales $\{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0\}$. We see that for A30 at around 120,000 grid cells we see the average time to update a grid cell go flat. Thus for the RTX 4060 GPU we would have to run for around 110,000 grid cells. For the RTX 4060 we would recommend around 400,000 grid cells. This asymptotic behavior happens because for small grid sizes we are not creating enough blocks to make apt use of the memory latency hiding. When it flattens all the sm are busy and no sm is idle.

3 Code

3.1 Naiv

```
/**
 * Shallow Water Equations Solver – CUDA Implementation
 *
 * Linear Shallow Water Equations Solver using staggered grid in space and in time:
 *
 *      / t + (Hu)/ x + (Hv)/ y = 0 (continuity)
 * u / t - fv = - g / x - r u (x-velocity)
 * v / t + fu = - g / y - r v (y-velocity)
 *
 * Variables are staggered as:
 * - (eta): Water surface elevation at cell centers (ny, nx)
 * - H: Water depth at rest at cell centers (ny, nx)
 * - u: x-velocity at cell faces (ny, nx+1)
 * - v: y-velocity at cell faces (ny+1, nx)
 *
 * We use a staggered leapfrog scheme where:
 * - is updated using u, v at the current time
 * - u, v are updated using at the new time
 *
 * This is equivalent to the scheme used in many operational ocean models and is stable f
 */

#include <iostream>
#include <fstream>
#include <vector>
#include <cmath>
#include <algorithm>
#include <string>
#include <chrono>
#include <cstring>
#include <cuda_runtime.h>

// Block size for CUDA kernels (you can experiment with different sizes for performance)
#define BLOCK_SIZE_X 16
#define BLOCK_SIZE_Y 16

// Precision control
#define PREC 8 // 4 for float, 8 for double

#if PREC == 4
    using real_t = float;
#elif PREC == 8
    using real_t = double;
#else
    using real_t = float;
#endif

// Helper macro for CUDA error checking
#define CUDA_CHECK(call) { \
```

```

        cudaError_t err = call;
        if (err != cudaSuccess) {
            fprintf(stderr, "CUDA-error-at-%s:%d-   -%s\n",
                __FILE__, __LINE__, cudaGetErrorString(err));
            exit(EXIT_FAILURE);
        }
    }

// 2D array class for convenient indexing
template<typename T>
class Array2D {
private:
    std::vector<T> data;
    int rows, cols;
public:
    Array2D() : rows(0), cols(0) {}
    Array2D(int ny, int nx) : rows(ny), cols(nx), data(ny * nx, T(0)) {}
    Array2D(int ny, int nx, T val) : rows(ny), cols(nx), data(ny * nx, val) {}

    T& operator()(int j, int i) { return data[j * cols + i]; }
    const T& operator()(int j, int i) const { return data[j * cols + i]; }

    int ny() const { return rows; }
    int nx() const { return cols; }
    T* ptr() { return data.data(); }
    const T* ptr() const { return data.data(); }

    void resize(int ny, int nx) {
        rows = ny;
        cols = nx;
        data.resize(ny * nx, T(0));
    }

    void fill(T val) {
        std::fill(data.begin(), data.end(), val);
    }

    Array2D<T> copy() const {
        Array2D<T> result(rows, cols);
        result.data = data;
        return result;
    }
};

// Configuration class for simulation parameters
class Sim_Configuration {
public:
    double scale = 1.0;
    double tend = 3600.0;
    double fout = 60.0;
    std::string filename = "output.dat";

    Sim_Configuration(int argc, char* argv[]) {
        int i = 1;

```

```

while (i < argc) {
    std::string arg = argv[i];
    if (arg == "-h") {
        std::cout << "Usage: ./krakatau --scale<scaling factor for grid resolution> --tend<duration in seconds between each snapshot> --out<name of output file>" << std::endl;
        exit(0);
    } else if (i == argc - 1) {
        throw std::runtime_error("The last argument (" + arg + ") must have a value");
    } else {
        if (arg == "--scale") {
            scale = std::stod(argv[i + 1]);
            if (scale <= 0) throw std::runtime_error("scale must be a positive real number");
        } else if (arg == "--tend") {
            tend = std::stod(argv[i + 1]);
            if (tend < 0) throw std::runtime_error("tend must be a positive real number");
        } else if (arg == "--fout") {
            fout = std::stod(argv[i + 1]);
            if (fout < 0) throw std::runtime_error("fout must be a positive real number");
        } else if (arg == "--out") {
            filename = argv[i + 1];
        } else {
            std::cout << "----> error: the argument type is not recognized" << std::endl;
        }
        i += 2;
    }
}

};

/**
 * Rescale the grid resolution by a scale factor using bilinear interpolation.
 */
void rescale_grid(double scale,
                  const Array2D<real_t>& H, const std::vector<real_t>& lon, const std::vector<real_t>& lat,
                  double dx, double dy,
                  Array2D<real_t>& new_H, std::vector<real_t>& new_lon, std::vector<real_t>& new_lat,
                  double& new_dx, double& new_dy, int& new_nx, int& new_ny) {

    int old_ny = H.ny();
    int old_nx = H.nx();

    new_nx = static_cast<int>(old_nx * scale);
    new_ny = static_cast<int>(old_ny * scale);

    new_dx = (dx * old_nx) / new_nx;
    new_dy = (dy * old_ny) / new_ny;

    double dlon = lon[1] - lon[0];
    double dlat = lat[1] - lat[0];

    double lon_min = lon[0] - dlon / 2;
    double lon_max = lon[old_nx - 1] + dlon / 2;
    double lat_min = lat[0] - dlat / 2;
    double lat_max = lat[old_ny - 1] + dlat / 2;

```

```

double new_dlon = (lon_max - lon_min) / new_nx;
double new_dlat = (lat_max - lat_min) / new_ny;

new_lon.resize(new_nx);
new_lat.resize(new_ny);
for (int i = 0; i < new_nx; i++) new_lon[i] = lon_min + 0.5 * new_dlon + i * new_dlon;
for (int j = 0; j < new_ny; j++) new_lat[j] = lat_min + 0.5 * new_dlat + j * new_dlat;

std::vector<double> x_new(new_nx), y_new(new_ny);
for (int i = 0; i < new_nx; i++) x_new[i] = (new_lon[i] - lon[0]) / dlon;
for (int j = 0; j < new_ny; j++) y_new[j] = (new_lat[j] - lat[0]) / dlat;

new_H.resize(new_ny, new_nx);
for (int j = 0; j < new_ny; j++) {
    int y0 = std::max(0, std::min(old_ny - 1, static_cast<int>(std::floor(y_new[j]))));
    int y1 = std::min(old_ny - 1, y0 + 1);
    double yf = std::max(0.0, std::min(1.0, y_new[j] - y0));

    for (int i = 0; i < new_nx; i++) {
        int x0 = std::max(0, std::min(old_nx - 1, static_cast<int>(std::floor(x_new[i]))));
        int x1 = std::min(old_nx - 1, x0 + 1);
        double xf = std::max(0.0, std::min(1.0, x_new[i] - x0));

        double row_y0 = H(y0, x0) * (1 - xf) + H(y0, x1) * xf;
        double row_y1 = H(y1, x0) * (1 - xf) + H(y1, x1) * xf;
        new_H(j, i) = row_y0 * (1 - yf) + row_y1 * yf;
    }
}
}

// =====
// CUDA kernels
// =====

// Step 1:  $\frac{1}{t} = - \frac{(Hu)}{x} - \frac{(Hv)}{y}$  (continuity equation)
// eta : (ny, nx)
// u : (ny, nx+1)
// v : (ny+1, nx)
// H_at_u: (ny, nx+1)
// H_at_v: (ny+1, nx)
__global__ void kernel_update_eta(real_t* eta, const real_t* u, const real_t* v,
                                const real_t* H_at_u, const real_t* H_at_v,
                                const char* land_mask,
                                real_t dx, real_t dy, real_t dt, int nx, int ny) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    if (i >= nx || j >= ny) return; // bounds check
    if (land_mask[j*nx + i]) {
        //eta[j*nx + i] = 0.0;
        return;
    }
};

```

```

    real_t flux_x_right = H_at_u[j*(nx+1) + (i+1)] * u[j*(nx+1) + (i+1)];
    real_t flux_x_left  = H_at_u[j*(nx+1) + i] * u[j*(nx+1) + i];
    real_t flux_y_top   = H_at_v[(j+1)*nx + i] * v[(j+1)*nx + i];
    real_t flux_y_bottom = H_at_v[j*nx + i] * v[j*nx + i];

    real_t dflux_x = (flux_x_right - flux_x_left) / dx;
    real_t dflux_y = (flux_y_top - flux_y_bottom) / dy;

    eta[j*nx + i] -= dt * (dflux_x + dflux_y);
}

// Update interior u faces (i=1..nx-1, j=0..ny-1)
// u_new : (ny, nx+1)
// wall_u : (ny, nx+1)
// eta : (ny, nx)
--global-- void kernel_interior_u(const real_t* d_eta, real_t* d_u_new,
                                const char* d_wall_u, const real_t* d_u,
                                int nx, int ny, real_t dx, real_t dy, real_t dt, real_t
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // Interior u faces only: i from 1 to nx-1
    if (i < 1 || i >= nx || j >= ny) return;

    real_t deta_dx = (d_eta[j*nx + i] - d_eta[j*nx + (i-1)]) / dx;

    d_u_new[j*(nx+1) + i] = d_u[j*(nx+1) + i] - dt * g * deta_dx;
    if (d_wall_u[j*(nx+1) + i]) d_u_new[j*(nx+1) + i] = 0.0;
}

// Update interior v faces (i=0..nx-1, j=1..ny-1)
// v_new : (ny+1, nx)
// wall_v : (ny+1, nx)
// eta : (ny, nx)
--global-- void kernel_interior_v(const real_t* d_eta, real_t* d_v_new,
                                const char* d_wall_v, const real_t* d_v,
                                int nx, int ny, real_t dx, real_t dy, real_t dt, real_t
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // Interior v faces only: j from 1 to ny-1
    if (i >= nx || j < 1 || j >= ny) return;

    real_t deta_dy = (d_eta[j*nx + i] - d_eta[(j-1)*nx + i]) / dy;
    d_v_new[j*nx + i] = d_v[j*nx + i] - dt * g * deta_dy;
    if (d_wall_v[j*nx + i]) d_v_new[j*nx + i] = 0.0;
}

// Open outflow boundary for u: zero gradient on left/right u-faces

```

```

// One thread per row (j), launched with 1D grid of size ny
// u_new: (ny, nx+1)
__global__ void kernel_outflow_u(real_t* d_u_new, int nx, int ny) {
    int j = blockIdx.x * blockDim.x + threadIdx.x;
    if (j >= ny) return;

    d_u_new[j*(nx+1) + 0] = d_u_new[j*(nx+1) + 1]; // left face
    d_u_new[j*(nx+1) + nx] = d_u_new[j*(nx+1) + (nx-1)]; // right face
}

// Open outflow boundary for v: zero gradient on bottom/top v-faces
// One thread per column (i), launched with 1D grid of size nx

__global__ void kernel_outflow_v(real_t* d_v_new, int nx, int ny) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= nx) return;
    d_v_new[0*nx + i] = d_v_new[1*nx + i]; // bottom face
    d_v_new[ny*nx + i] = d_v_new[(ny-1)*nx + i]; // top face
}

__global__ void kernel_boundaries(
    real_t* __restrict__ eta,
    real_t* __restrict__ u_new,
    real_t* __restrict__ v_new,
    int nx, int ny)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // eta top/bottom
    if (idx < nx) {
        eta[0 * nx + idx] = eta[1 * nx + idx];
        eta[(ny-1) * nx + idx] = eta[(ny-2) * nx + idx];
    }
    // eta left/right
    if (idx < ny) {
        eta[idx*nx + 0] = eta[idx*nx + 1];
        eta[idx*nx + (nx-1)] = eta[idx*nx + (nx-2)];
    }
    // u outflow left/right faces
    if (idx < ny) {
        u_new[idx*(nx+1) + 0] = u_new[idx*(nx+1) + 1];
        u_new[idx*(nx+1) + nx] = u_new[idx*(nx+1) + (nx-1)];
    }
    // v outflow bottom/top faces
    if (idx < nx) {
        v_new[0 * nx + idx] = v_new[1 * nx + idx];
        v_new[ny * nx + idx] = v_new[(ny-1) * nx + idx];
    }
}

```

```

/**
 * Advance solution by one time step.
 */
void step(real_t* d_eta, real_t*& d_u, real_t*& d_v,
          real_t*& d_u_new, real_t*& d_v_new,           // pre-allocated scratch buffers
          real_t g, const real_t* d_H_at_u, const real_t* d_H_at_v,
          const char* d_land_mask, const char* d_wall_u, const char* d_wall_v,
          real_t dx, real_t dy, real_t dt, int nx, int ny) {

    // 2D grid covering (ny, nx) cell centres
    dim3 block(BLOCK_SIZE_X, BLOCK_SIZE_Y);
    dim3 grid_eta((nx + BLOCK_SIZE_X - 1) / BLOCK_SIZE_X,
                  (ny + BLOCK_SIZE_Y - 1) / BLOCK_SIZE_Y);

    // Step 1: update eta (continuity equation)
    kernel_update_eta<<<grid_eta, block>>>(d_eta, d_u, d_v,
                                             d_H_at_u, d_H_at_v, d_land_mask,
                                             dx, dy, dt, nx, ny);

    int blockld = BLOCK_SIZE_X * BLOCK_SIZE_Y;    // 256 threads per block

    // Step 2: update u      grid covers (ny, nx+1) faces
    dim3 grid_u((nx+1 + BLOCK_SIZE_X - 1) / BLOCK_SIZE_X,
                (ny + BLOCK_SIZE_Y - 1) / BLOCK_SIZE_Y);
    kernel_interior_u<<<grid_u, block>>>(d_eta, d_u_new, d_wall_u, d_u,
                                         nx, ny, dx, dy, dt, g);
    kernel_outflow_u<<<(ny + blockld - 1) / blockld, blockld>>>(d_u_new, nx, ny);
    // Step 3: update v      grid covers (ny+1, nx) faces
    dim3 grid_v((nx + BLOCK_SIZE_X - 1) / BLOCK_SIZE_X,
                (ny+1 + BLOCK_SIZE_Y - 1) / BLOCK_SIZE_Y);
    kernel_interior_v<<<grid_v, block>>>(d_eta, d_v_new, d_wall_v, d_v,
                                         nx, ny, dx, dy, dt, g);

    kernel_outflow_v<<<(nx + blockld - 1) / blockld, blockld>>>(d_v_new, nx, ny);

    // Open boundary conditions on eta
    int bc_n = std::max(nx, ny);
    kernel_boundaries<<<(bc_n + blockld - 1) / blockld, blockld>>>(
        d_eta, d_u_new, d_v_new, nx, ny);

    // Pointer swap
    std::swap(d_u, d_u_new);
    std::swap(d_v, d_v_new);
}

/**
 * Shallow Water Solver class
 */
class ShallowWaterSolver {
public:
    Array2D<real_t> H;

```

```

std::vector<real_t> lon, lat;
real_t dx, dy;
real_t cfl;
int ny, nx;
real_t g = 9.81;

Array2D<char> land_mask;
Array2D<real_t> H_at_u; // (ny, nx+1)
Array2D<real_t> H_at_v; // (ny+1, nx)
Array2D<char> land_u; // (ny, nx+1)
Array2D<char> land_v; // (ny+1, nx)
Array2D<char> wall_u; // (ny, nx+1)
Array2D<char> wall_v; // (ny+1, nx)

Array2D<real_t> eta; // (ny, nx)
Array2D<real_t> u; // (ny, nx+1)
Array2D<real_t> v; // (ny+1, nx)

real_t dt;

ShallowWaterSolver(const Array2D<real_t>& elevation,
                   const std::vector<real_t>& lon_in, const std::vector<real_t>& lat_in,
                   real_t dx_in, real_t dy_in, real_t cfl_in = 0.5)
: lon(lon_in), lat(lat_in), dx(dx_in), dy(dy_in), cfl(cfl_in) {

ny = elevation.ny();
nx = elevation.nx();

H.resize(ny, nx);
for (int j = 0; j < ny; j++)
    for (int i = 0; i < nx; i++)
        H(j, i) = std::max(-elevation(j, i), 0.0);

land_mask.resize(ny, nx);
for (int j = 0; j < ny; j++)
    for (int i = 0; i < nx; i++)
        land_mask(j, i) = (H(j, i) < 1.0);

H_at_u.resize(ny, nx + 1);
for (int j = 0; j < ny; j++) {
    H_at_u(j, 0) = H(j, 0);
    for (int i = 1; i < nx; i++) H_at_u(j, i) = 0.5 * (H(j, i-1) + H(j, i));
    H_at_u(j, nx) = H(j, nx-1);
}

H_at_v.resize(ny + 1, nx);
for (int i = 0; i < nx; i++) {
    H_at_v(0, i) = H(0, i);
    for (int j = 1; j < ny; j++) H_at_v(j, i) = 0.5 * (H(j-1, i) + H(j, i));
    H_at_v(ny, i) = H(ny-1, i);
}

land_u.resize(ny, nx + 1);
for (int j = 0; j < ny; j++)

```

```

        for (int i = 0; i < nx + 1; i++)
            land_u(j, i) = (H_at_u(j, i) < 1.0);

land_v.resize(ny + 1, nx);
for (int j = 0; j < ny + 1; j++)
    for (int i = 0; i < nx; i++)
        land_v(j, i) = (H_at_v(j, i) < 1.0);

wall_u.resize(ny, nx + 1);
wall_u.fill(false);
for (int j = 0; j < ny; j++)
    for (int i = 1; i < nx; i++)
        wall_u(j, i) = land_mask(j, i-1) || land_mask(j, i);

wall_v.resize(ny + 1, nx);
wall_v.fill(false);
for (int j = 1; j < ny; j++)
    for (int i = 0; i < nx; i++)
        wall_v(j, i) = land_mask(j-1, i) || land_mask(j, i);

eta.resize(ny, nx);    eta.fill(0.0);
u.resize(ny, nx + 1);  u.fill(0.0);
v.resize(ny + 1, nx);  v.fill(0.0);

real_t max_depth = 0;
for (int j = 0; j < ny; j++)
    for (int i = 0; i < nx; i++)
        max_depth = std::max(max_depth, H(j, i));
real_t c_max = std::sqrt(g * max_depth);
dt = cfl * std::min(dx, dy) / c_max;

int land_count = 0;
for (int j = 0; j < ny; j++)
    for (int i = 0; i < nx; i++)
        if (land_mask(j, i)) land_count++;

std::cout << "Shallow-Water-Solver-initialized:" << std::endl;
std::cout << "--Grid-size:-" << ny << "-x-" << nx << std::endl;
std::cout << "--Grid-spacing:-dx=" << dx << "m,-dy=" << dy << "m" << std::endl;
std::cout << "--Max-depth:-" << max_depth << "m,-Max-wave-speed:-" << c_max << "m/s" << std::endl;
std::cout << "--Time-step:-" << dt << "s-(CFL=" << cfl << ")" << std::endl;
std::cout << "--Land-cells:-" << land_count << "-(" << 100.0 * land_count / (ny * nx) << "%)" << std::endl;
}

void set_gaussian_initial_condition(int i_center, int j_center,
                                   real_t amplitude = 20.0, real_t spread = 20.0) {
    for (int j = 0; j < ny; j++) {
        for (int i = 0; i < nx; i++) {
            real_t dx_ic = (i - j_center) / spread;
            real_t dy_ic = (j - i_center) / spread;
            real_t r2 = dx_ic * dx_ic + dy_ic * dy_ic;
            eta(j, i) = amplitude * std::exp(-0.5 * r2);
            if (land_mask(j, i)) eta(j, i) = 0.0;
        }
    }
}

```

```

    }
    std::cout << "Gaussian-IC:-center=(" << i_center << "," << j_center
        << " ),-amplitude=" << amplitude << "m,-spread=" << spread << "-cells" <
}

void run(double duration, double output_interval,
        std::vector<real_t>& times, std::vector<real_t>& performance,
        std::vector<Array2D<real_t>>& etas) {

    int n_steps = static_cast<int>(duration / dt);
    int output_every = output_interval > 0 ?
        std::max(1, static_cast<int>(output_interval / dt)) :
        std::max(1, n_steps / 100);

    std::cout << "Running:-" << duration << "s-(" << n_steps << "-steps),-output-every-
        << output_every * dt << "s" << std::endl;

    // Allocate GPU arrays

    real_t *d_eta, *d_u, *d_v;
    real_t *d_H_at_u, *d_H_at_v;
    char *d_land_mask, *d_wall_u, *d_wall_v;
    real_t *d_u_new, *d_v_new;

    CUDA_CHECK(cudaMalloc(&d_eta, ny * nx * sizeof(real_t)));
    CUDA_CHECK(cudaMalloc(&d_u, ny * (nx+1) * sizeof(real_t)));
    CUDA_CHECK(cudaMalloc(&d_v, (ny+1) * nx * sizeof(real_t)));
    CUDA_CHECK(cudaMalloc(&d_H_at_u, ny * (nx+1) * sizeof(real_t)));
    CUDA_CHECK(cudaMalloc(&d_H_at_v, (ny+1) * nx * sizeof(real_t)));
    CUDA_CHECK(cudaMalloc(&d_land_mask, ny * nx * sizeof(char)));
    CUDA_CHECK(cudaMalloc(&d_wall_u, ny * (nx+1) * sizeof(char)));
    CUDA_CHECK(cudaMalloc(&d_wall_v, (ny+1) * nx * sizeof(char)));
    CUDA_CHECK(cudaMalloc(&d_u_new, ny * (nx+1) * sizeof(real_t)));
    CUDA_CHECK(cudaMalloc(&d_v_new, (ny+1) * nx * sizeof(real_t)));

    CUDA_CHECK(cudaMemcpy(d_eta, eta.ptr(), ny * nx * sizeof(real_t),
        cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_u, u.ptr(), ny * (nx+1) * sizeof(real_t),
        cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_v, v.ptr(), (ny+1) * nx * sizeof(real_t),
        cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_H_at_u, H_at_u.ptr(), ny * (nx+1) * sizeof(real_t),
        cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_H_at_v, H_at_v.ptr(), (ny+1) * nx * sizeof(real_t),
        cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_land_mask, land_mask.ptr(), ny * nx * sizeof(char),
        cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_wall_u, wall_u.ptr(), ny * (nx+1) * sizeof(char),
        cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_wall_v, wall_v.ptr(), (ny+1) * nx * sizeof(char),
        cudaMemcpyHostToDevice));

    auto tlast = std::chrono::high_resolution_clock::now();
    long long nupdated = 0;

    for (int n = 0; n < n_steps; n++) {
        step(d_eta, d_u, d_v,
            d_u_new, d_v_new,
            g, d_H_at_u, d_H_at_v,

```

```

        d_land_mask, d_wall_u, d_wall_v,
        dx, dy, dt, nx, ny);
nupdated += nx * ny;

if (n % output_every == 0) {
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(eta.ptr(), d_eta, ny * nx * sizeof(real_t), cudaMemcpyDeviceToHost));

    auto tnow = std::chrono::high_resolution_clock::now();
    double elapsed = std::chrono::duration<double>(tnow - tlast).count();
    performance.push_back(1e9 * elapsed / nupdated);
    nupdated = 0;
    times.push_back(n * dt);
    etas.push_back(eta.copy());
    tlast = tnow;

    if (n % (output_every) == 0) {
        real_t max_eta = 0;
        for (int j = 0; j < ny; j++)
            for (int i = 0; i < nx; i++)
                max_eta = std::max(max_eta, std::abs(eta(j, i)));
        std::cout << "--Step--" << n << "/" << n_steps << ",-t=" << n * dt
                    << "s,-max| |= " << max_eta << "m" << std::endl;
    }
}

// Final output if needed
if (nupdated > 0) {
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(eta.ptr(), d_eta, ny * nx * sizeof(real_t), cudaMemcpyDeviceToHost));
    auto tnow = std::chrono::high_resolution_clock::now();
    double elapsed = std::chrono::duration<double>(tnow - tlast).count();
    performance.push_back(1e9 * elapsed / nupdated);
    times.push_back(n_steps * dt);
    etas.push_back(eta.copy());
}

CUDA_CHECK(cudaFree(d_eta));
CUDA_CHECK(cudaFree(d_u));
CUDA_CHECK(cudaFree(d_v));
CUDA_CHECK(cudaFree(d_H_at_u));
CUDA_CHECK(cudaFree(d_H_at_v));
CUDA_CHECK(cudaFree(d_land_mask));
CUDA_CHECK(cudaFree(d_wall_u));
CUDA_CHECK(cudaFree(d_wall_v));
CUDA_CHECK(cudaFree(d_u_new));
CUDA_CHECK(cudaFree(d_v_new));

std::cout << "Simulation-complete!" << std::endl;
}
};

```

```

void simulate(const Sim_Configuration& config) {
    std::string input_file = "./bathymetry_krakatau.bin";

    std::ifstream fin(input_file, std::ios::binary);
    if (!fin) throw std::runtime_error("Cannot open input file: " + input_file);

    int32_t nx, ny;
    fin.read(reinterpret_cast<char*>(&nx), sizeof(int32_t));
    fin.read(reinterpret_cast<char*>(&ny), sizeof(int32_t));

    std::vector<double> lon_d(nx), lat_d(ny);
    fin.read(reinterpret_cast<char*>(lon_d.data()), nx * sizeof(double));
    fin.read(reinterpret_cast<char*>(lat_d.data()), ny * sizeof(double));

    std::vector<double> elevation_d(nx * ny);
    fin.read(reinterpret_cast<char*>(elevation_d.data()), nx * ny * sizeof(double));
    fin.close();

    std::vector<real_t> lon(nx), lat(ny);
    for (int i = 0; i < nx; i++) lon[i] = lon_d[i];
    for (int j = 0; j < ny; j++) lat[j] = lat_d[j];

    Array2D<real_t> elevation(ny, nx);
    for (int j = 0; j < ny; j++)
        for (int i = 0; i < nx; i++)
            elevation(j, i) = elevation_d[j * nx + i];

    double dlon = std::abs(lon[1] - lon[0]);
    double dlat = std::abs(lat[1] - lat[0]);
    real_t dx = dlon * 111000;
    real_t dy = dlat * 111000;

    Array2D<real_t> new_elevation;
    std::vector<real_t> new_lon, new_lat;
    real_t new_dx, new_dy;
    int new_nx, new_ny;
    rescale_grid(config.scale, elevation, lon, lat, dx, dy,
                 new_elevation, new_lon, new_lat, new_dx, new_dy, new_nx, new_ny);

    elevation = new_elevation;
    lon = new_lon; lat = new_lat;
    dx = new_dx; dy = new_dy;
    nx = new_nx; ny = new_ny;

    const real_t krakatau_lon = 105.423;
    const real_t krakatau_lat = -6.102;

    int j_krakatau = 0, i_krakatau = 0;
    real_t min_lon_diff = std::abs(lon[0] - krakatau_lon);
    real_t min_lat_diff = std::abs(lat[0] - krakatau_lat);
    for (int i = 1; i < nx; i++) {
        if (std::abs(lon[i] - krakatau_lon) < min_lon_diff) {
            min_lon_diff = std::abs(lon[i] - krakatau_lon);
            j_krakatau = i;
        }
    }
}

```

```

    }
}
for (int j = 1; j < ny; j++) {
    if (std::abs(lat[j] - krakatau_lat) < min_lat_diff) {
        min_lat_diff = std::abs(lat[j] - krakatau_lat);
        i_krakatau = j;
    }
}

ShallowWaterSolver solver(elevation, lon, lat, dx, dy, 0.4);
solver.set_gaussian_initial_condition(i_krakatau, j_krakatau, 40.0, 10);

std::vector<real_t> times, performance;
std::vector<Array2D<real_t>> etas;
solver.run(config.tend, config.fout, times, performance, etas);

if (performance.size() > 1) {
    double perf_sum = 0.0;
    for (size_t k = 1; k < performance.size(); k++) perf_sum += performance[k];
    std::cout << "\nAverage performance is -"
                << perf_sum / (performance.size() - 1)
                << " ns/grid point update" << std::endl;
}

int nframes = times.size();
std::cout << "\nSaving results to -" << config.filename << "-" << nframes
            << " frames, -" << nx << "x" << ny << "-nx-x-ny-grid" << std::endl;

std::ofstream fout(config.filename, std::ios::binary);
int32_t nframes_i = nframes, nx_i = nx, ny_i = ny;
fout.write(reinterpret_cast<char*>(&nframes_i), sizeof(int32_t));
fout.write(reinterpret_cast<char*>(&nx_i), sizeof(int32_t));
fout.write(reinterpret_cast<char*>(&ny_i), sizeof(int32_t));

std::vector<float> lon_f(nx), lat_f(ny);
for (int i = 0; i < nx; i++) lon_f[i] = lon[i];
for (int j = 0; j < ny; j++) lat_f[j] = lat[j];
fout.write(reinterpret_cast<char*>(lon_f.data()), nx * sizeof(float));
fout.write(reinterpret_cast<char*>(lat_f.data()), ny * sizeof(float));

std::vector<float> times_f(nframes), perf_f(nframes);
for (int i = 0; i < nframes; i++) { times_f[i] = times[i]; perf_f[i] = performance[i]; }
fout.write(reinterpret_cast<char*>(times_f.data()), nframes * sizeof(float));
fout.write(reinterpret_cast<char*>(perf_f.data()), nframes * sizeof(float));

std::vector<float> elev_f(nx * ny);
for (int j = 0; j < ny; j++)
    for (int i = 0; i < nx; i++)
        elev_f[j * nx + i] = elevation(j, i);
fout.write(reinterpret_cast<char*>(elev_f.data()), nx * ny * sizeof(float));

std::vector<float> eta_f(nx * ny);
for (int frame = 0; frame < nframes; frame++) {
    for (int j = 0; j < ny; j++)

```

```

        for (int i = 0; i < nx; i++)
            eta_f[j * nx + i] = etas[frame](j, i);
        fout.write(reinterpret_cast<char*>(eta_f.data()), nx * ny * sizeof(float));
    }
    fout.close();
}

int main(int argc, char* argv[]) {
    auto tstart = std::chrono::high_resolution_clock::now();

    Sim_Configuration config(argc, argv);
    simulate(config);
    auto tstop = std::chrono::high_resolution_clock::now();
    std::cout << (tstop - tstart).count()*1e-9 << std::endl;
}

```